

Testes Automatizados

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br

Sumário

- Introdução aos Testes Automatizados
- Pipeline de Testes
- Testes Unitários
- Testes de Integração
- Testes de Performance
- Qualidade e Validação Automatizada
- Atividade

Estrutura de pipeline

- Pipeline: Sequência de validações automatizadas executadas durante o ciclo de entrega do software.
- Build uma única vez
 - O mesmo artefato é promovido entre ambientes
- Mesmo deploy em todos os ambientes
 - Desenvolvimento → Homologação → Produção
- Falhou? Para tudo
 - Nenhuma etapa continua após erro

Um desenvolvedor altera a tela de login de um sistema e envia o código para o GitHub. A partir disso, o pipeline inicia automaticamente:

1. O sistema baixa o código atualizado
2. Compila a aplicação
3. Executa testes automáticos
4. Verifica a qualidade do código
5. Gera uma nova versão da aplicação
6. Pública em ambiente de homologação

O que são Testes Automatizados?

- São verificações executadas automaticamente por ferramentas para validar o comportamento esperado de uma aplicação.
- Permitem verificar funcionalidades sem depender da execução manual de cada cenário por parte do desenvolvedor.
- Podem ser executados durante o desenvolvimento e integrados às pipelines de Integração Contínua (CI).
- Quando uma alteração é realizada, os testes podem ser executados automaticamente para identificar possíveis falhas antes da entrega do software.

Por que automatizar os testes?

- **Reduzir erros humanos:** Diminuir a dependência de verificações manuais e repetitivas.
- **Aumentar a velocidade:** Executar diversas verificações em poucos segundos ou minutos.
- **Identificar falhas rapidamente:** Detectar problemas logo após uma alteração no código.
- **Garantir consistência:** Executar os mesmos cenários de teste sempre que necessário.
- **Aumentar a confiabilidade:** Verificar continuamente se as funcionalidades existentes continuam funcionando.

Pipeline de testes

- O pipeline de testes executa automaticamente validações no sistema após o processo de build.
- O objetivo é garantir que novas alterações não quebrem funcionalidades existentes do sistema.
- Principais funções
 - Execução automática de testes
 - Validação contínua da aplicação
 - Identificação rápida de falhas

Pipeline de qualidade

- O pipeline de qualidade realiza verificações automatizadas para avaliar se o código atende aos padrões definidos antes de avançar para as próximas etapas.
- O objetivo é identificar problemas de qualidade, inconsistências e possíveis falhas no código de forma antecipada.

GitHub Actions Marketplace

- É o catálogo de Actions, ferramentas e extensões que podem ser utilizadas nos workflows do GitHub Actions.
- Permite encontrar soluções prontas para build, testes, qualidade, segurança, deploy, cache e outras etapas da pipeline.
- As Actions podem ser desenvolvidas pelo GitHub, parceiros ou pela comunidade.
- GitHub Marketplace: <https://github.com/marketplace?type=actions&category=testing>

Situação-Problema



Vendedores de uma loja de roupas realizavam o cálculo de descontos manualmente, gerando **erros frequentes e inconsistências no atendimento**. A falta de padronização resultava em clientes insatisfeitos, retrabalho e perda de confiança no processo de vendas.

Erros humanos

Cálculos incorretos geravam divergências nos valores cobrados

Inconsistência

Cada vendedor aplicava descontos de forma diferente

Situação-Problema

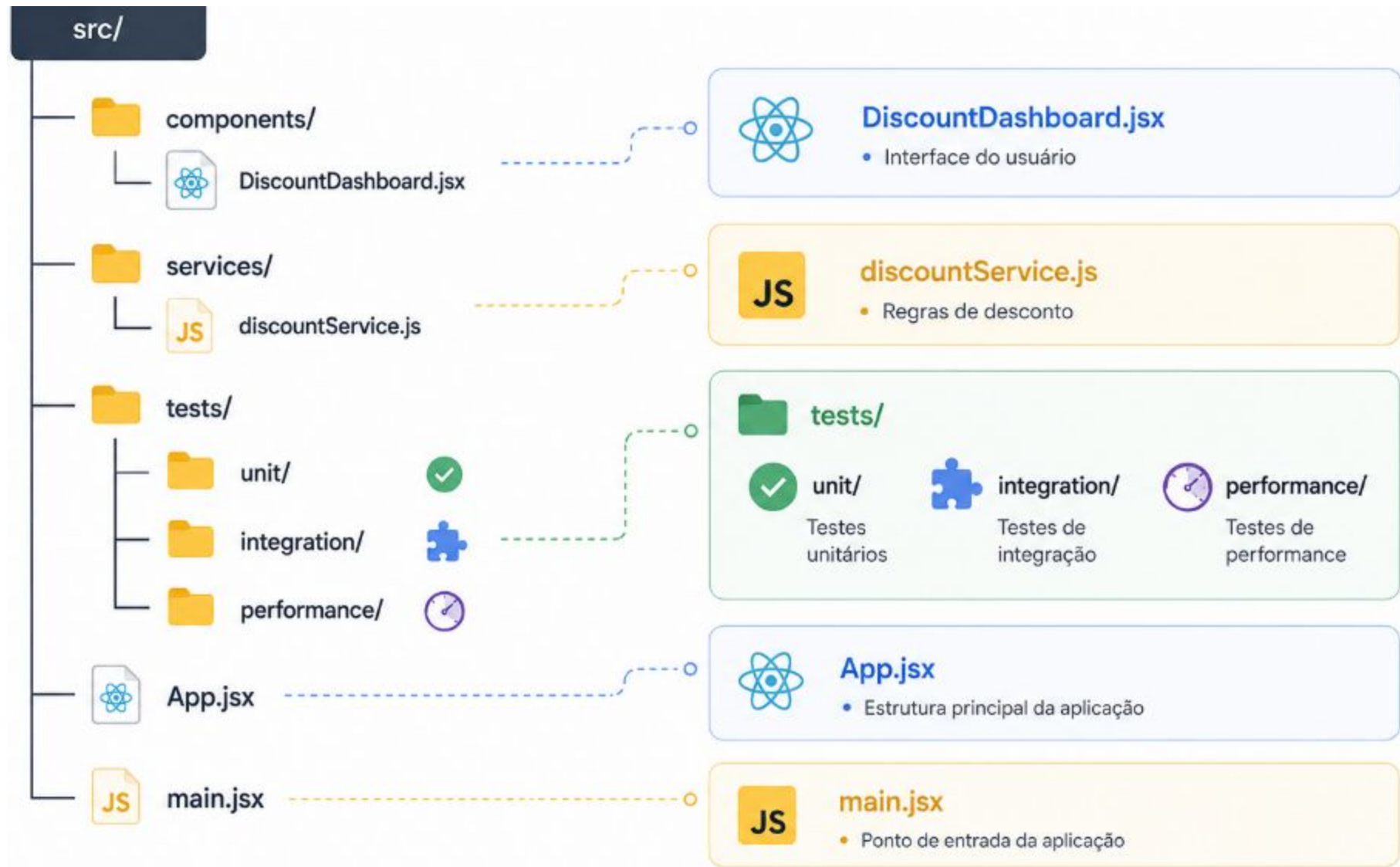
Foi desenvolvido um **sistema web** capaz de calcular descontos automaticamente, seguindo regras definidas pela empresa de forma **padronizada** e **confiável**. A automação reduziu falhas, agilizou o atendimento aos clientes e eliminou a dependência de cálculos manuais durante as vendas.



The screenshot shows a web interface titled "Sistema Comercial" with the subtitle "Cálculo automático de descontos". Below the title, there is a label "Valor da Compra" above a text input field containing the placeholder "Digite o valor". At the bottom of the form is a blue button labeled "Calcular Desconto".

Fonte: Elaborado pelo autor.

- Compras **abaixo** de R\$ 100 recebem **10%** de desconto.
- Compras **iguais ou superiores** a R\$ 100 recebem **5%** de desconto.



Fonte: <https://github.com/devisontakatu/projeto-pipeline-testes>

Tipos de Testes

Os diferentes tipos de testes técnicos atuam em conjunto para garantir qualidade, confiabilidade, estabilidade e o correto funcionamento do software, além de verificar seu alinhamento aos requisitos e objetivos definidos. Para este projeto, serão considerados os três tipos de testes a seguir:



Unitários

Validam funções e métodos de forma isolada.



Integração

Verificam a comunicação eficiente entre os módulos.



Performance

Avaliam a velocidade e estabilidade sob carga.

- Pires e Militão (2019).

Testes Unitários

- Responsáveis pela validação de pequenas partes do sistema, como funções, métodos e componentes isolados do código.
- São executados de forma rápida e automatizada, permitindo identificar falhas durante o desenvolvimento antes da integração completa da aplicação.
- Facilitam a manutenção do software, aumentam a confiabilidade das funcionalidades e reduzem custos com correções futuras.

Exemplo Prático: Alterando a regra de desconto para 15%.

Problema: O sistema esperava desconto de 10% ou 5%.

Testes de Integração

- Testam a comunicação e o funcionamento conjunto entre módulos, serviços, APIs e bancos de dados da aplicação.
- Permitem identificar falhas na troca de informações entre componentes que funcionam corretamente de forma isolada.
- Garantem maior estabilidade do sistema, validando fluxos completos e reduzindo riscos de inconsistências em ambientes de produção.

Exemplo Prático: Alterando o texto do botão para "Calcular".

Problema: Não encontrou o botão "Calcular Desconto" durante a interação.

Testes de Performance

- Responsáveis por avaliar o comportamento da aplicação em situações de carga, tempo de resposta e consumo de recursos.
- Permitem identificar gargalos, lentidão e falhas relacionadas ao desempenho do sistema antes da implantação em produção.
- Garantem estabilidade, escalabilidade e melhor experiência do usuário, validando se a aplicação suporta operações dentro do tempo esperado.

Exemplo Prático: Alterando o tempo máximo esperado da resposta para 1ms.

Problema: A aplicação não conseguiu responder dentro do tempo configurado.

Conceito de Qualidade

- Qualidade de software representa a capacidade do sistema atender requisitos, necessidades do usuário e objetivos do negócio com eficiência.
- Um software de qualidade deve apresentar confiabilidade, estabilidade, facilidade de manutenção e capacidade de evolução ao longo do tempo.
- Em ambientes DevOps, a qualidade deixa de ser responsabilidade exclusiva dos testes e passa a integrar todo o ciclo de desenvolvimento, entrega e operação.

“Qualidade deve fazer parte do processo” HUMBLE E PRIKLANDNICKI (2013).

ESLint: Verificador de Qualidade

O ESLint é uma ferramenta de análise estática utilizada para identificar problemas de padronização, qualidade e possíveis inconsistências em códigos JavaScript. No contexto de DevOps e Integração Contínua, o ESLint atua como uma validação automatizada dentro da pipeline, auxiliando no controle de qualidade e impedindo a integração de códigos fora dos padrões definidos pelo projeto (Moraes, 2015).

Exemplo Prático: Adicionando `const x=1;`

Problema: Variável foi criada e nunca utilizada (`no-unused-vars`).

O que acontece quando um teste falha?

Quando uma etapa da pipeline identifica um problema, o processo pode ser interrompido para evitar que uma versão inválida avance para as próximas etapas. **Principais consequências:**

- A alteração não avança para as próximas etapas.
- A equipe recebe um feedback sobre o problema.
- O código pode ser corrigido antes da entrega.
- Reduz-se o risco de levar uma versão com falhas para outro ambiente.

Situação-Problema: Conclusão



Com a adoção de práticas de DevOps e Integração Contínua, a aplicação tornou-se mais confiável, segura e preparada para mudanças constantes.

A automação de testes, validações e builds permitiu reduzir falhas, aumentar a qualidade das entregas e acelerar o desenvolvimento continuamente.



Testes Unit.
Garantia de
lógica individual



Integração
Validação entre
módulos



Performance
Monitoramento
de carga



ESLint
Qualidade e
padronização

Atividade

Criem um novo projeto que integre três tipos de testes automatizados: unitário, integração e performance. Organizem o projeto para que cada tipo de teste possa ser executado e analisado de forma independente.

Em seguida, criem uma pipeline de Integração Contínua (CI) utilizando um arquivo `.yml` no GitHub Actions, configurada para executar automaticamente a cada push na branch main. A pipeline deverá instalar as dependências, executar os três tipos de testes e apresentar os resultados.

Referências

- HUMBLE J; PRIKLANDNICKI R. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013
- MUNIZ, A.; et al. Jornada Devops: Unindo Cultura Ágil, Lean e Tecnologia Para Entrega De Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. DevOps na prática: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. Entrega contínua em Android: Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.
- ARUNDEL, J. DOMINGUS, J. DevOps nativo de nuvem com Kubernetes. São Paulo: Novatec, 2019.

Referências

- MORAES, G. Caixa de Ferramentas DevOps: Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015.
- PIRES, A.; MILITÃO, J. Integração Contínua com Jenkins. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. Descomplicando o Docker. 2 ed. São Paulo: Brasport, 2018.
- SILVERMAN, R. E. Git: guia prático. São Paulo: Novatec, 2019.
- KIM, G; HUMBLE, J; DEBOIS, P; WILLIS, J; Manual de DEVOPS: Como obter agilidade, confiabilidade e segurança em organizações tecnológicas. São Paulo: Starlin Alta Editora, 2018.

Testes Automatizados

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br